

**CENTRO REGIONAL UNIVERSITARIO
CÓRDOBA - IUA**



Trabajo Final

"Compilador de C"

Desarrollo de Herramientas de Software – Ingeniería en Informática

Profesor

Eschoyez, Maximiliano A.

Alumno

Calvo, Tomás

tcalvo@alumnos.iua.edu.ar

Junio de 2026

Introducción y Problemática Abordada

El objetivo de este Trabajo Final para la materia "Desarrollo de Herramientas de Software" consiste en la implementación de un compilador para un subconjunto del lenguaje C. La problemática central radica en transformar código fuente de alto nivel en una representación intermedia que permita verificar la corrección gramatical y lógica del programa, preparándolo para fases posteriores de compilación.

Características Principales

- Análisis léxico y sintáctico generado por ANTLR4
- Tabla de símbolos con soporte de scopes anidados (pila de contextos)
- Validación semántica: declaración, inicialización, tipos, doble declaración
- Generación de TAC para todas las estructuras del lenguaje: declaraciones, asignaciones, if/else, while, for y funciones
- Parámetros de funciones manejados con modelo push/pop (param / pop)
- Manejo de scoping de variables: renombre con sufijo de nivel (_0, _1...) para evitar colisiones entre contextos
- Optimización de TAC: propagación de constantes y eliminación de subexpresiones repetidas
- Bloqueo de generación de TAC cuando existen errores semánticos o sintácticos
- Comparación antes/después de la optimización con métricas de reducción

Herramientas Utilizadas

- **Python:** Lenguaje anfitrión para toda la lógica del compilador.
- **ANTLR4:** Herramienta utilizada para generar el analizador léxico (Lexer) y sintáctico (Parser) a partir del archivo de gramática compiler.g4.
- **Git:** Control de versiones y gestión del proyecto.

Desarrollo de la Solución Propuesta

Arquitectura del Compilador

El compilador sigue una arquitectura modular con separación clara de responsabilidades. Cada fase está implementada en un módulo independiente que se comunica a través de interfaces bien definidas.

Diagrama de módulos

```
App.py                                (orquestador)
|-- compilerLexer.py                  (ANTLR4 - análisis léxico)
|-- compilerParser.py                (ANTLR4 - análisis sintáctico)
|-- Escucha.py                       (Listener - análisis semántico + errores)
|   |-- TABLA.py                     (TS: Variable, Funcion, Contexto, TS Singleton)
|-- caminante.py                     (Visitor base de recorrido del árbol)
|-- CodigoIntermedio.py              (Visitor - generación TAC + scoping)
'-- optimizador.py                   (optimización de TAC)
```

Patrones de diseño utilizados

- **Singleton:** La clase TS (Tabla de Símbolos) usa una instancia única global accedida mediante TS.getInstance().
- **Listener:** Escucha.py implementa compiladorListener para recibir eventos enter/exit de cada nodo del AST durante el recorrido con ParseTreeWalker.
- **Visitor:** CodigoIntermedio.py implementa compiladorVisitor para recorrer el AST de forma controlada y retornar valores desde cada nodo.

Flujo de ejecución

1. 1. Lexer → tokens (análisis léxico)
2. 2. Parser → AST (análisis sintáctico, errores capturados por Escucha)
3. 3. ParseTreeWalker + Escucha → análisis semántico y construcción de tabla de símbolos
4. 4. if tiene_errores(): STOP (bloqueo de TAC)
5. 5. CodigoIntermedio.visit(AST) → TAC original
6. 6. Optimizador → TAC optimizado
7. 7. Reporte de comparación y escritura de codigo_tres_direcciones.txt

Análisis Léxico

El análisis léxico se define en la gramática compiler.g4 y es generado automáticamente por ANTLR4. El lexer establece los tokens y las reglas de producción que definen la estructura jerárquica del lenguaje.

Categorías de Tokens

Categoría	Tokens	Ejemplo
Agrupadores	PA PC LLA LLC CA CC PYC COMA	(){}[]; ,
Aritméticos	ASIG SUMA RESTA MULT DIV MOD	= + - * / %
Relacionales	MENOR MAYOR MENOREQ MAYOREQ EQUAL NEQUAL	< > <= >= == !=
Incr./Decr.	INCREMENTO DECREMENTO	++ --
Literales	NUMERO DECIMAL	42 3.14
Tipos	INT DOUBLE FLOAT	int double float
Control	IF ELSE FOR WHILE RETURN	if else for while return
Identificadores	ID	x foo miVar

Análisis Sintáctico

La gramática compiler.g4 define un lenguaje imperativo con tipos estáticos basados en C. Mediante ANTLR4 se generó un analizador sintáctico (Parser LL(*)) que procesa los tokens y construye un árbol de sintaxis concreta (Parse Tree).

En esta etapa el Parser verifica:

- Balanceo de llaves {}, corchetes [] y paréntesis ()
- Correcta terminación de instrucciones con punto y coma ';'

- Estructura y precedencia válida de operaciones aritméticas y lógicas
- Correcta formación gramatical de estructuras de control (if/else, while, for) y de funciones

Los errores sintácticos son interceptados por Escucha, que implementa ErrorListener de ANTLR4, y se reportan con línea y columna exactas. Si hay errores sintácticos, la compilación se detiene antes de la fase semántica.

Tabla de Símbolos

La tabla de símbolos (TS) almacena información sobre todos los identificadores declarados: variables y funciones. Está implementada en TABLA.py con las clases ID, Variable, Funcion, Contexto y TS.

Gestión de pila de contextos

Los contextos (scopes) se implementan como una pila de objetos Contexto. Al entrar a un bloque (función, if, while, for), se hace push de un nuevo contexto con addContexto(). Al salir, se hace pop con delContexto(). La búsqueda de símbolos recorre la pila desde el tope hacia la base, implementando shadowing de variables.

Clases de la TS

Clase	Descripción
ID	Clase base con nombre, tipo, estado de inicialización y uso
Variable	Hereda de ID, representa una variable declarada
Funcion	Hereda de ID, agrega lista de parámetros (tipo, nombre)
Contexto	Diccionario de símbolos para un scope
TS (Singleton)	Pila de contextos activos + contextos cerrados para reporte

Funciones y modelo de pila de parámetros

Cuando se declara una función, Escucha.py:

8. Registra la función en el scope padre (permite recursión y visibilidad externa).
9. Abre un scope hijo (push de contexto) para el cuerpo de la función.
10. Registra los parámetros en el scope hijo como variables inicializadas.
11. Al salir de la función (exitFuncion), cierra el scope hijo.

Análisis Semántico

El análisis semántico se realiza en Escucha.py (Listener). Se ejecuta durante el recorrido del AST por el ParseTreeWalker y valida la corrección lógica del programa más allá de la sintaxis.

Validaciones implementadas

Validación	Dónde se detecta	Tipo
Variable no declarada	exitAsignacion, exitFactor	Error
Variable no inicializada	exitFactor	Error
Doble declaración	exitDeclaracion, enterFuncion	Error
Tipos incompatibles en asignación	exitAsignacion	Error
Llamada a función no declarada	exitCall	Error
Cantidad incorrecta de argumentos	exitCall	Error
Tipo incorrecto de argumento	exitCall	Error
Variable declarada pero no usada	exitPrograma	Advertencia
Función declarada pero no usada	exitPrograma	Advertencia

Las advertencias no bloquean la generación de TAC. Solo los errores sintácticos y semánticos detienen el pipeline.

Gestión de Errores y Advertencias

Escucha.py centraliza la recolección y reporte de todos los errores y advertencias del compilador, actuando como único punto de gestión de estado de errores.

Categorías

Categoría	Método	Bloquea TAC?
Error sintáctico	reportar_error_sintactico(linea, col, msg)	Sí
Error semántico	reportar_error_semantico(linea, msg)	Sí
Advertencia	reportar_advertencia(linea, msg)	No

Flujo de errores

- Los errores sintácticos se capturan mediante el método `syntaxError()` que Escucha hereda de `ErrorListener` de ANTLR4.
- Los errores semánticos se agregan durante el walk del listener.
- `tiene_errores()` verifica si existen errores (no cuenta advertencias).
- Si hay errores, se imprime el reporte y se bloquea la generación de TAC con un mensaje informativo.
- Las advertencias de variables no usadas se emiten siempre en `exitPrograma`, independientemente de si hay errores.

Generación de Código Intermedio (TAC)

El código de tres direcciones (TAC) es una representación intermedia donde cada instrucción tiene como máximo un operador y tres operandos. Se genera mediante CodigoIntermedio.py, que implementa el patrón Visitor.

Formato de instrucciones TAC

Tipo	Formato	Ejemplo
Asignación simple	var = valor	x_0 = 5
Operación binaria	t = a op b	t0 = x_0 + 3
Declaración sin init	DECLARE var	DECLARE y_0
Salto condicional	if not cond goto L	if not t0 goto L1
Salto incondicional	goto L	goto L0
Etiqueta	L:	L0:
Parámetro	param valor	param x_0
Llamada	t = call func	t0 = call sumar
Retorno	return valor	return t0
Función inicio	FUNC nombre:	FUNC sumar:
Función fin	END FUNC nombre	END FUNC sumar
Pop parámetro	pop nombre	pop a_1

Manejo de Scoping en el TAC

CodigoIntermedio.py implementa su propia pila de contextos (pila_contextos) independiente de la tabla de símbolos semántica. Cada variable recibe un sufijo de nivel (_0 para el contexto global, _1 para el primer nivel anidado, etc.) que garantiza que colisiones de nombres entre scopes no generen código incorrecto.

Ejemplo: si existe `int x = 100` globalmente y `int x = 99` dentro de un if, el TAC generará `x_0 = 100` y `x_1 = 99` respectivamente, preservando la semántica.

Generación para expresiones

Las expresiones se evalúan bottom-up. Cada subexpresión genera un temporal (t0, t1, ...) que se usa como operando en la instrucción siguiente.

```
Entrada:  x + y * 3
TAC:
    t0 = y_0 * 3
    t1 = x_0 + t0
```

Generación para if/else

```
if (x > 0) { y = 1; } else { y = 0; }
TAC:
    t0 = x_0 > 0
    if not t0 goto L0
```

```

    y_0 = 1
    goto L1
L0:
    y_0 = 0
L1:

```

Generación para while

```

while (i < 10) { i++; }
TAC:
L0:
    t0 = i_0 < 10
    if not t0 goto L1
    t1 = i_0 + 1
    i_0 = t1
    goto L0
L1:

```

Generación para for

El for se descompone en: inicialización + etiqueta + condición + cuerpo + incremento + salto.

```

for (int i = 0; i < 5; i++) { x = x + i; }
TAC:
    i_1 = 0
L0:
    t0 = i_1 < 5
    if not t0 goto L1
    t1 = x_0 + i_1
    x_0 = t1
    t2 = i_1 + 1
    i_1 = t2
    goto L0
L1:

```

Generación para funciones

Las funciones usan el modelo push/pop. Al llamar, los argumentos se pasan con 'param' (equivalente a push). Al definir la función, los parámetros se reciben con pop. Cada parámetro se registra con su sufijo de nivel para evitar colisiones con variables globales del mismo nombre.

```

int sumar(int a, int b) { return a + b; }
int r = sumar(3, 5);
TAC:
FUNC sumar:
    pop a_1
    pop b_1
    t0 = a_1 + b_1
    return t0
END FUNC sumar
param 3
param 5
t1 = call sumar
r_0 = t1

```

Optimización de Código TAC

El módulo optimizador.py implementa dos pasadas de optimización que se ejecutan iterativamente hasta alcanzar un punto fijo (sin más cambios) o un máximo de 10 iteraciones. Cada pasada opera sobre la lista de instrucciones TAC como strings.

1) Propagación de Constantes

Cuando una variable o temporal se define como una constante numérica, se reemplaza cada uso posterior de esa variable por el valor constante directamente. Además, si la expresión resultante es operable con valores numéricos conocidos, se evalúa con eval() de Python (constant folding).

Antes	Después
x_0 = 5 / t0 = x_0 * 2	x_0 = 5 / t0 = 5 * 2 → t0 = 10
t1 = 3 + 4	t1 = 7

Protecciones: las instrucciones de control (if, goto, FUNC, END, param, pop, return) no se propagan. Las etiquetas limpian la tabla de constantes conocidas para no propagar valores fuera de su región de código.

2) Eliminación de Subexpresiones Repetidas

Si una expresión ya fue calculada y asignada a un temporal anterior, los temporales que repiten esa misma expresión se reemplazan por el temporal original. Esto elimina cálculos redundantes.

Antes: <pre> t0 = b * b t1 = 4 * a t2 = t1 * c t3 = b * b ← repetido </pre>	Después: <pre> t0 = b * b t1 = 4 * a t2 = t1 * c t3 = t0 ← reutiliza t0 </pre>
--	---

Protección importante: las llamadas a función (call nombre) nunca se consideran subexpresiones repetidas, ya que los argumentos pasados (param) pueden ser distintos en cada llamada aunque el nombre de la función sea el mismo.

Iteración hasta punto fijo

Las dos pasadas se ejecutan en orden dentro de un bucle. Si ninguna realiza cambios, el optimizador termina. Esto es necesario porque la propagación de constantes puede crear oportunidades para la eliminación de repetidas en la siguiente iteración.

Reporte y comparación

El optimizador genera dos salidas:

- **Reporte:** lista detallada de cada optimización aplicada con formato 'Tipo: antes → después'.
- **Comparación:** métricas de líneas originales, optimizadas, eliminadas y porcentaje de reducción.

Resumen de Archivos del Proyecto

Archivo	Líneas	Descripción
compiler.g4	~209	Gramática ANTLR4 del lenguaje
App.py	~59	Punto de entrada, orquestador del pipeline
Escucha.py	~407	Listener: análisis semántico, errores y advertencias
TABLA.py	~110	Tabla de Símbolos: Variable, Funcion, Contexto, TS (Singleton)
CodigoIntermedio.py	~597	Visitor: generación de TAC con scoping por sufijos
optimizador.py	~174	Propagación de constantes y eliminación de repetidas
caminante.py	~8	Visitor base de recorrido del árbol
compilerLexer.py	auto	Generado por ANTLR4
compilerParser.py	auto	Generado por ANTLR4
compilerListener.py	auto	Generado por ANTLR4
compilerVisitor.py	auto	Generado por ANTLR4

Conclusión

El desarrollo de este compilador permitió comprender la complejidad real que existe detrás de la traducción de lenguajes de programación. El uso de ANTLR4 facilitó significativamente la etapa de parsing, permitiendo enfocar los esfuerzos en la lógica semántica y la generación de código intermedio.

Uno de los mayores desafíos técnicos fue la correcta implementación del manejo de scopes en el generador de TAC. La decisión de mantener una pila de contextos propia en CodigoIntermedio.py, independiente de la tabla de símbolos semántica, permitió resolver correctamente la colisión de nombres entre variables de distintos niveles de anidamiento, incluyendo el caso crítico de parámetros de función con el mismo nombre que variables globales.

La implementación del optimizador dejó en evidencia la importancia de contemplar correctamente los límites de cada técnica: en particular, la necesidad de excluir las llamadas a funciones de la eliminación de subexpresiones repetidas, dado que los argumentos pasados pueden variar entre llamadas aunque la función sea la misma.

Referencias

- Repositorio GitHub: <https://github.com/tomas-calvo01/DHS2025-calvo->